INNOVATORS PCCOER

Github Link – https://github.com/DishaMusale/NutriAI **[Click to view Github Repo]**

NutriAI Demo Video Link (Google Drive) - https://drive.google.com/file/d/1vlxPiq-im8SuRpfyvnnjA-B7SfaF8-mI/view?usp=sharing **[Click to view demo video]**

Team Members
1) Sana Nair
2) Disha Musale
3) Riya Musmade
4) Kadambari Shendage
5) Rutuja Biradar

Brief about NutriAI-

NutriAI – AI Powered Smart Nutrition Planner

NutriAI is an intelligent nutrition planning platform that generates personalized meal plans based on a user's health goals, dietary preferences, lifestyle, activity level, and budget. The system collects user information through a structured onboarding questionnaire and uses AI-driven logic to create customized weekly nutrition recommendations.

The platform incorporates multiple AI-inspired processing stages, including user profiling, constraint validation, dietary preference analysis, meal optimization, and explainable plan generation. By considering factors such as weight goals, diet type, allergies, medical conditions, cooking time, and budget constraints, NutriAI delivers practical and personalized meal suggestions tailored to individual needs.

Key Features

- Secure user authentication using Clerk
- Multi-step personalized nutrition questionnaire
- AI-based meal plan generation
- Goal-focused recommendations (Weight Loss, Muscle Gain, Maintenance)
- Budget-aware meal planning
- Dietary restriction and allergy consideration
- Interactive dashboard displaying generated meal plans
- Modern and responsive user interface

Technology Stack

- Frontend: React.js, CSS, Vite
- Backend: Python, FastAPI
- Authentication: Clerk
- Version Control: Git & GitHub

Objective

The primary objective of NutriAI is to make personalized nutrition planning accessible, affordable, and convenient by leveraging AI to create meal plans that align with users' unique health goals and lifestyle requirements.

1. App.jsx

Root application component with routing and Clerk authentication

```
import {
SignedIn,
SignedOut,
SignInButton,
SignUpButton,
UserButton,
} from "@clerk/clerk-react";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import LandingPage from "./components/LandingPage";
import QuestionnairePage from "./pages/QuestionnairePage";
import DashboardPage from "./pages/DashboardPage";
function App() {
return (
<BrowserRouter>
<Routes>
<Route
path="/"
element={
<div>
<SignedOut>
<LandingPage />
<div
style={{
textAlign: "center",
marginTop: "-100px",
}}
>
<SignInButton />
<SignUpButton />
</div>
</SignedOut>
<SignedIn>
<div
style={{
position: "fixed",
top: "20px",
right: "20px",
zIndex: 1000
}}
>
<UserButton />
</div>
<LandingPage />
</SignedIn>
</div>
}
/>
<Route
path="/questionnaire"
element={<QuestionnairePage />}
```

```
/>
<Route
path="/dashboard"
element={<DashboardPage />}
/>
</Routes>
</BrowserRouter>
);
}
export default App;
```

2. Questionnaire.jsx

7-step onboarding form with validation and Firestore integration

```
import { useState } from "react"
import { useNavigate } from "react-router-dom"
import { db } from "../firebase";
import { collection, addDoc } from "firebase/firestore";
import "./Questionnaire.css"
function Questionnaire() {
const navigate = useNavigate()
const [step, setStep] = useState(1)
const [formData, setFormData] = useState({
name: "",
age: "",
gender: "",
height: "",
weight: "",
goal: "",
targetWeight: "",
activityLevel: "",
proteinPriority: "",
dietType: "",
cuisine: "",
foodsEnjoy: "",
foodsAvoid: "",
allergies: [],
medicalConditions: [],
budget: "",
cookingTime: "",
mealsPerDay: "",
optimizationPreference: "",
waterIntake: "",
})
const handleChange = (e) => {
const { name, value } = e.target
setFormData(prev => ({ ...prev, [name]: value }))
}
const handleCheckbox = (field, value) => {
setFormData(prev => ({
...prev,
[field]: prev[field].includes(value)
? prev[field].filter(v => v !== value)
: [...prev[field], value]
}))
}
const handleNext = () => {
if (step === 1 && (!formData.name || !formData.age)) {
alert("Please fill Name and Age before continuing")
return
}
if (step === 2 && !formData.goal) {
alert("Please select your Primary Goal before continuing")
return
}
```

```
if (step === 4 && (
formData.allergies.length === 0 ||
formData.medicalConditions.length === 0
)
) {
alert("Please complete all required fields");
return;
}
setStep(prev => prev + 1)
window.scrollTo(0, 0)
}
const handleBack = () => {
setStep(prev => prev - 1)
window.scrollTo(0, 0)
}
const handleSubmit = async () => {
try {
localStorage.setItem(
"userprofile",
JSON.stringify(formData)
);
if (!formData.waterIntake) {
alert("Please complete all required fields");
return;
}
await addDoc(
collection(db, "users"),
formData
);
navigate("/dashboard");
} catch (error) {
console.error(error);
}
};
return (
<div  className="questionnaire-container">
{/* Progress Bar */}
<div  className="progress-bar-container">
<div className="progress-bar" style={{ width: `${(step / 7) * 100}%` }}></div>
</div>
<p className="progress-text">Step {step} of 7</p>
{/* Section 1 */}
{step === 1 && (
<div className="section">
<div className="section-header">
<div>
<h2 className="section-title">
Personal Information
</h2>
</div>
<p className="section-subtitle">
Fill in to personalize your plan
```

```
</p>
</div>
<div className="form-grid">
<div className="question">
<label>Name</label>
<input
type="text"
name="name"
value={formData.name}
onChange={handleChange}
placeholder="Your name"
/>
</div>
<div className="question">
<label>Age</label>
<input
type="number"
name="age"
value={formData.age}
onChange={handleChange}
placeholder="e.g. 28"
/>
</div>
<div className="question">
<label>Height (cm)</label>
<input
type="number"
name="height"
value={formData.height}
onChange={handleChange}
placeholder="e.g. 165"
/>
</div>
<div className="question">
<label>Weight (kg)</label>
<input
type="number"
name="weight"
value={formData.weight}
onChange={handleChange}
placeholder="e.g. 62"
/>
</div>
<div className="question full-width">
<label>Gender</label>
<div className="checkbox-group">
{["Male","Female","Other"].map(option => (
<label
key={option}
className={
formData.gender === option
? "selected"
: ""
```

```
}
>
<input
type="radio"
name="gender"
value={option}
checked={formData.gender === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
</div>
</div>
)}
{/* Section 2 */}
{step === 2 && (
<div className="section">
<div className="section-header">
<div>
<h2 className="section-title">Fitness Goals</h2>
</div>
</div>
<div className="form-grid">
<div className="question full-width">
<label>Primary Goal</label>
<div className="radio-group">
{["Weight Loss", "Muscle Gain", "Maintenance", "General Health"].map(option => (
<label
key={option}
className={formData.goal === option ? "selected" : ""}
>
<input
type="radio"
name="goal"
value={option}
checked={formData.goal === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
<div className="question">
<label>Target Weight (kg)</label>
<input
type="number"
name="targetWeight"
value={formData.targetWeight}
onChange={handleChange}
placeholder="e.g. 55"
```

```
/>
</div>
<div className="question">
<label>Protein Priority</label>
<div className="radio-group">
{["High Protein", "Balanced Nutrition", "No Preference"].map(option => (
<label
key={option}
className={formData.proteinPriority === option ? "selected" : ""}
>
<input
type="radio"
name="proteinPriority"
value={option}
checked={formData.proteinPriority === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
<div className="question full-width">
<label>Activity Level</label>
<div className="radio-group">
{["Sedentary", "Lightly Active", "Moderately Active", "Very Active"].map(option => (
<label
key={option}
className={formData.activityLevel === option ? "selected" : ""}
>
<input
type="radio"
name="activityLevel"
value={option}
checked={formData.activityLevel === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
</div>
</div>
)}
{/* Section 3 */}
{step === 3 && (
<div className="section">
<div className="section-header">
<div>
<h2 className="section-title">Dietary Preferences</h2>
</div>
</div>
```

```
<div className="form-grid">
<div className="question full-width">
<label>Diet Type *</label>
<div className="radio-group">
{["Vegetarian", "Non-Vegetarian", "Vegan", "Jain"].map(option => (
<label
key={option}
className={formData.dietType === option ? "selected" : ""}
>
<input
type="radio"
name="dietType"
value={option}
checked={formData.dietType === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
<div className="question full-width">
<label>Preferred Cuisine *</label>
<div className="radio-group">
{["Marathi", "North Indian", "South Indian", "Gujarati", "Punjabi", "Jain", "No
Preference"].map(option => (
<label
key={option}
className={formData.cuisine === option ? "selected" : ""}
>
<input
type="radio"
name="cuisine"
value={option}
checked={formData.cuisine === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
<div className="question">
<label>Foods You Enjoy (Optional)</label>
<textarea
name="foodsEnjoy"
value={formData.foodsEnjoy}
onChange={handleChange}
placeholder="e.g. Paneer, Dosa, Rajma"
rows={4}
/>
</div>
<div className="question">
<label>Foods To Avoid (Optional)</label>
```

```
<textarea
name="foodsAvoid"
value={formData.foodsAvoid}
onChange={handleChange}
placeholder="e.g. Mushrooms, Eggs, Spicy food"
rows={4}
/>
</div>
</div>
</div>
)}
{/* Section 4 */}
{step === 4 &&
(
<div className="section">
<div className="section-header">
<div>
<h2 className="section-title">Allergies & Restrictions</h2>
</div>
</div>
<div className="form-grid">
<div className="question full-width">
<label>Food Allergies *</label>
<div className="checkbox-group">
{["Peanuts", "Dairy", "Gluten", "Soy", "Seafood", "None"].map(option => (
<label
key={option}
className={
formData.allergies.includes(option)
? "selected"
: ""
}
>
<input
type="checkbox"
checked={formData.allergies.includes(option)}
onChange={() =>
handleCheckbox("allergies",  option)
}
/>
{option}
</label>
))}
</div>
</div>
<div className="question full-width">
<label>Medical Conditions *</label>
<div className="checkbox-group">
{["Diabetes", "Hypertension", "Thyroid", "PCOS", "None"].map(option => (
<label
key={option}
className={
formData.medicalConditions.includes(option)
```

```
? "selected"
: ""
}
>
<input
type="checkbox"
checked={formData.medicalConditions.includes(option)}
onChange={() =>
handleCheckbox("medicalConditions",  option)
}
/>
{option}
</label>
))}
</div>
</div>
</div>
</div>
)}
{/* Section 5 */}
{step === 5 && (
<div className="section">
<div className="section-header">
<div>
<span className="step-label">STEP 5 OF 7</span>
<h2 className="section-title">Budget & Lifestyle</h2>
</div>
</div>
<div className="form-grid">
<div className="question full-width">
<label>Monthly Food Budget *</label>
<div className="radio-group">
{[
"Less than ■2000",
"■2000 - ■4000",
"■4000 - ■6000",
"More than ■6000"
].map(option => (
<label
key={option}
className={formData.budget === option ? "selected" : ""}
>
<input
type="radio"
name="budget"
value={option}
checked={formData.budget === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
```

```
</div>
<div className="question">
<label>Daily Cooking Time *</label>
<div className="radio-group">
{[
"Less than 15 minutes",
"15-30 minutes",
"30-60 minutes",
"More than 60 minutes"
].map(option => (
<label
key={option}
className={formData.cookingTime === option ? "selected" : ""}
>
<input
type="radio"
name="cookingTime"
value={option}
checked={formData.cookingTime === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
<div className="question">
<label>Meals Per Day *</label>
<div className="radio-group">
{["3 Meals", "4 Meals", "5 Meals"].map(option => (
<label
key={option}
className={formData.mealsPerDay === option ? "selected" : ""}
>
<input
type="radio"
name="mealsPerDay"
value={option}
checked={formData.mealsPerDay === option}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
</div>
</div>
)}
{/* Section 6 */}
{step === 6 && (
<div className="section">
<div className="section-header">
```

```
<div>
<span className="step-label">STEP 6 OF 7</span>
<h2 className="section-title">Optimization Preference</h2>
</div>
</div>
<div className="form-grid">
<div className="question full-width">
<label>What should NutriAI prioritize? *</label>
<div className="radio-group">
{[
"Budget Optimized",
"Nutrition Optimized",
"Balanced Approach"
].map(option => (
<label
key={option}
className={
formData.optimizationPreference === option
? "selected"
: ""
}
>
<input
type="radio"
name="optimizationPreference"
value={option}
checked={
formData.optimizationPreference === option
}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
</div>
</div>
)}
{/* Section 7 */}
{step === 7 && (
<div className="section">
<div className="section-header">
<div>
<span className="step-label">STEP 7 OF 7</span>
<h2 className="section-title">Lifestyle Information</h2>
</div>
</div>
<div className="form-grid">
<div className="question full-width">
<label>Daily Water Intake *</label>
<div className="radio-group">
{[
```

```
"Less than 1L",
"1-2L",
"2-3L",
"More than 3L"
].map(option => (
<label
key={option}
className={
formData.waterIntake === option
? "selected"
: ""
}
>
<input
type="radio"
name="waterIntake"
value={option}
checked={
formData.waterIntake === option
}
onChange={handleChange}
/>
{option}
</label>
))}
</div>
</div>
</div>
</div>
)}
{/* Navigation Buttons */}
<div className="btn-row">
{step > 1 && (
<button className="back-btn" onClick={handleBack}>← Back</button>
)}
{step < 7 && (
<button className="next-btn" onClick={handleNext}>Next →</button>
)}
{step === 7 && (
<button className="submit-btn" onClick={handleSubmit}>
Submit & Generate My Plan →
</button>
)}
</div>
</div>
)
}
export default Questionnaire
```

3. Dashboard.jsx

AI meal-plan generator with profile display and day-card renderer

```
import { useState } from "react";
import { db } from "../firebase";
import { collection, addDoc } from "firebase/firestore";
import "./Dashboard.css";
function Dashboard() {
const profile = JSON.parse(
localStorage.getItem("userprofile")
);
const [mealPlan, setMealPlan] = useState("");
const [loading, setLoading] = useState(false);
const generatePlan = async () => {
try {
setLoading(true);
const response = await fetch(
"http://127.0.0.1:8000/generate-plan",
{
method: "POST",
headers: {
"Content-Type": "application/json",
},
body: JSON.stringify(profile),
}
);
const data = await response.json();
setMealPlan(data.meal_plan);
try {
await addDoc(
collection(db, "mealPlans"),
{
userName: profile?.name || "Unknown",
goal: profile?.goal || "",
dietType: profile?.dietType || "",
budget: profile?.budget || "",
activityLevel: profile?.activityLevel || "",
mealPlan: data.meal_plan,
createdAt: new Date(),
}
);
console.log("Meal plan saved successfully!");
} catch (firestoreError) {
console.error(
"Firestore save failed:",
firestoreError
);
}
} catch (error) {
console.error(error);
} finally {
setLoading(false);
```

```
}
};
const parseMealPlan = (text) => {
if (!text) return [];
const matches = text.match(
/=== DAY \d+ ===[\s\S]*?(?=(=== DAY \d+ ===|$))/g
);
return matches || [];
};
const parsedDays = parseMealPlan(mealPlan);
console.log(parsedDays[0]);
return (
<div  className="dashboard-container">
<div  className="dashboard-content">
<div className="welcome-section">
<h1>Welcome {profile?.name} </h1>
<p>
Let's create a personalized nutrition plan for you.
</p>
</div>
<div className="profile-grid">
<div className="profile-card">
<h3>Goal</h3>
<p>{profile?.goal}</p>
</div>
<div className="profile-card">
<h3>Diet Type</h3>
<p>{profile?.dietType}</p>
</div>
<div className="profile-card">
<h3>Budget</h3>
<p>{profile?.budget}</p>
</div>
<div className="profile-card">
<h3>Activity Level</h3>
<p>{profile?.activityLevel}</p>
</div>
</div>
<button
className="generate-btn"
onClick={generatePlan}
>
Generate AI Meal Plan
</button>
{loading && (
<div className="loading-box">
Creating your personalized meal plan...
</div>
)}
{mealPlan && (
<div className="meal-section">
<h2>Your Personalized 7-Day Meal Plan</h2>
```

```
{parsedDays.map((day, index) => {
const lines = day.split("\n").filter((line) => line.trim());
return (
<div className="day-card" key={index}>
<h3>Day {index + 1}</h3>
{lines
.filter(line => !line.includes("=== DAY"))
.map((line, i) => (
<p key={i}>{line}</p>
))}
</div>
);
})}
</div>
)}
</div>
</div>
);
}
export default Dashboard;
```

4. main.py

FastAPI backend with Gemini 2.5 Flash meal-plan generation

```
from fastapi import FastAPI
from pydantic import BaseModel
import google.generativeai as genai
from dotenv import load_dotenv
import os
from fastapi.middleware.cors import CORSMiddleware
load_dotenv()
genai.configure(
api_key=os.getenv("GEMINI_API_KEY")
)
app = FastAPI()
app.add_middleware(
CORSMiddleware,
allow_origins=["*"],
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)
class UserProfile(BaseModel):
name: str
goal: str
dietType: str
budget: str
activityLevel: str
proteinPriority: str
allergies: list[str] = []
@app.post("/generate-plan")
def generate_plan(profile: UserProfile):
prompt = f"""
Create a personalized 7-day Indian meal plan.
User Details:
Name: {profile.name}
Goal: {profile.goal}
Diet Type: {profile.dietType}
Budget: {profile.budget}
Activity Level: {profile.activityLevel}
Protein Priority: {profile.proteinPriority}
IMPORTANT:
Return ONLY in this format:
=== DAY 1 ===
Breakfast: ...
Lunch: ...
Dinner: ...
Snack: ...
=== DAY 2 ===
Breakfast: ...
Lunch: ...
Dinner: ...
Snack: ...
Continue until DAY 7.
```

```
Rules:
- No introduction
- No conclusion
- No markdown
- No bullet points
- No explanations
- Only meal plans
- Use affordable Indian foods
- Keep meals practical
- Respect diet type
- Match the user's goal
"""
model = genai.GenerativeModel(
"models/gemini-2.5-flash"
)
response = model.generate_content(prompt)
return {
"meal_plan": response.text
}
```

5. firebase.js

Firebase app initialisation and Firestore export

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";
const firebaseConfig = {
apiKey: "AIzaSyBUqLK7AcyatcTEyPS_4H8UTzlRLx057Ss",
authDomain: "nutriai-19cec.firebaseapp.com",
projectId: "nutriai-19cec",
storageBucket: "nutriai-19cec.firebasestorage.app",
messagingSenderId: "414537310830",
appId: "1:414537310830:web:e3f84b717a6deec4ce7c7f",
measurementId: "G-7QNXKGR1T2"
};
const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
```